



Ecole Supérieure Privée d'Ingénierie et de
Communication

Rapport de Projet

Bibliothèque Intelligente avec Chatbot IA

Projet semestriel – Python

Étudiant	Ben Ammar Younes
Classe	TC1 G7
Année universitaire	2025–2026
Enseignant	Bilel Amdouni

Table des matières

1	Contexte et objectifs du projet	3
2	Analyse des besoins	5
2.1	Besoins fonctionnels	5
2.2	Besoins non fonctionnels	5
3	Technologies choisies et justification	7
3.1	Backend : Flask	7
3.2	Base de données : PostgreSQL	7
3.3	Frontend : CustomTkinter	8
3.4	Intégration de l'intelligence artificielle avec Google Gemini	8
3.5	Gestion de configuration et sécurité	9
4	Architecture globale de l'application	10
4.1	Vue d'ensemble de l'architecture	10
4.2	Description des composants	10
4.2.1	Frontend	10
4.2.2	Backend	11
4.2.3	Base de données	11
4.2.4	Module d'intelligence artificielle	11
4.3	Schéma architectural	11
4.4	Conclusion de l'architecture	12
5	Conception de la base de données	13
5.1	Modèle de données	13
5.2	Structure de la table <code>books</code>	13

5.3	Exemples de requêtes SQL utilisées	13
6	Fonctionnalités développées	15
6.1	Gestion CRUD des livres	15
6.2	Interface graphique	15
7	Intégration du chatbot IA	16
7.1	Présentation du chatbot	16
7.2	Architecture du système conversationnel	16
7.3	Communication avec la base de données	16
7.4	Construction du prompt contextuel	16
7.5	Exemples de scénarios pris en charge	18
8	Captures d'écran et démonstration	19
8.1	Tableau de bord principal	19
8.2	Ajout d'un livre	20
8.3	Modification d'un livre	21
8.4	Chatbot IA	22
9	Conclusion	24
10	Perspectives d'amélioration	25

1. Contexte et objectifs du projet

Dans le cadre du projet semestriel du module Python, il a été demandé de concevoir et de développer une application de gestion de bibliothèque intégrant des fonctionnalités d'intelligence artificielle. L'objectif principal est de mettre en pratique les concepts étudiés durant le semestre, notamment le développement d'applications Python, la gestion de bases de données, la conception d'interfaces graphiques et l'intégration d'API externes.

Les bibliothèques modernes nécessitent des outils informatiques permettant d'assurer une gestion efficace des ouvrages disponibles, de faciliter les opérations de recherche et de fournir une assistance rapide aux utilisateurs. Dans cette optique, le projet réalisé vise à développer une solution complète capable de gérer le catalogue des livres tout en offrant une interaction intelligente grâce à un chatbot basé sur l'intelligence artificielle.

L'application développée, intitulée *Bibliothèque Intelligente avec Chatbot IA*, repose sur une architecture client-serveur composée d'une interface graphique développée avec **CustomTkinter**, d'une API backend réalisée avec **Flask** et d'une base de données **PostgreSQL**. L'assistant conversationnel utilise l'API **Google Gemini** afin de répondre aux questions des utilisateurs en s'appuyant sur les données réelles enregistrées dans la bibliothèque.

Les objectifs principaux du projet sont les suivants :

- Concevoir une interface graphique moderne et ergonomique permettant la gestion des livres.
- Mettre en place une base de données relationnelle pour le stockage des informations relatives aux ouvrages.
- Développer les opérations CRUD (*Create, Read, Update, Delete*) nécessaires à l'administration du catalogue.
- Permettre la recherche rapide d'ouvrages selon différents critères.
- Intégrer un chatbot intelligent capable de comprendre des questions formulées en langage naturel.
- Connecter le chatbot à la base de données afin qu'il fournisse des réponses basées sur le contenu réel de la bibliothèque.
- Mettre en œuvre une architecture logicielle modulaire facilitant la maintenance et l'évolution du système.

À travers ce projet, plusieurs compétences techniques ont été mobilisées, notamment la programmation Python avancée, le développement d'interfaces graphiques, la conception de bases de données relationnelles, la création d'API REST ainsi que l'intégration de services d'intelligence artificielle. Le résultat obtenu constitue une application fonctionnelle capable de répondre aux exigences définies dans le cahier des charges tout en offrant une expérience utilisateur moderne et intuitive.

2. Analyse des besoins

L'analyse des besoins constitue une étape essentielle dans le développement de toute application. Elle permet d'identifier les fonctionnalités attendues par les utilisateurs ainsi que les contraintes techniques que le système doit respecter. Dans le cadre de ce projet, les besoins ont été divisés en deux catégories : les besoins fonctionnels et les besoins non fonctionnels.

2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les services que l'application doit fournir aux utilisateurs.

L'application doit permettre :

- L'ajout d'un nouveau livre dans la bibliothèque.
- L'affichage de la liste complète des livres enregistrés.
- La modification des informations relatives à un livre existant.
- La suppression d'un livre du catalogue.
- La consultation des informations détaillées d'un ouvrage.
- La gestion du statut d'un livre (Disponible, Emprunté ou Réservé).
- La communication avec un chatbot intelligent via une interface dédiée.
- L'interprétation des questions formulées en langage naturel.
- La consultation de la base de données par le chatbot afin de fournir des réponses précises et contextualisées.
- La recommandation d'ouvrages présents dans la bibliothèque selon les demandes de l'utilisateur.

2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et les caractéristiques techniques du système.

L'application doit respecter les exigences suivantes :

- **Ergonomie** : proposer une interface graphique moderne, intuitive et facile à utiliser.
- **Performance** : fournir des temps de réponse rapides lors des opérations CRUD et des recherches.
- **Fiabilité** : garantir l'intégrité des données stockées dans la base PostgreSQL.
- **Maintenabilité** : adopter une architecture modulaire facilitant l'ajout de nouvelles fonctionnalités.
- **Extensibilité** : permettre l'intégration future de nouvelles fonctionnalités sans refonte majeure du système.
- **Sécurité** : protéger les informations sensibles telles que les paramètres de connexion à la base de données et les clés d'API grâce à l'utilisation de variables d'environnement.
- **Compatibilité** : assurer le fonctionnement de l'application sur les systèmes d'exploitation compatibles avec Python.
- **Disponibilité** : garantir l'accès aux fonctionnalités principales de gestion de bibliothèque même en cas d'indisponibilité temporaire du chatbot.

L'ensemble de ces besoins a servi de base à la conception et au développement de l'application. Ils ont également guidé les choix technologiques effectués tout au long du projet.

3. Technologies choisies et justification

Le choix des technologies utilisées dans ce projet a été réalisé en tenant compte de plusieurs critères, notamment la simplicité de développement, la performance, la maintenabilité et la compatibilité avec les objectifs du cahier des charges. Les technologies retenues permettent de construire une application moderne, modulaire et facilement évolutive.

3.1 Backend : Flask

Pour le développement de l'API backend, le framework Flask a été retenu.

Flask est un micro-framework Python léger et flexible permettant de développer rapidement des applications web et des services REST. Son architecture minimaliste offre une grande liberté dans l'organisation du projet tout en conservant une excellente lisibilité du code.

Les principales raisons ayant motivé ce choix sont :

- Simplicité de prise en main et de développement.
- Architecture légère adaptée aux projets de taille moyenne.
- Facilité de création d'API REST.
- Intégration simple avec PostgreSQL.
- Large communauté et documentation abondante.

Dans cette application, Flask est utilisé comme couche intermédiaire entre l'interface graphique et la base de données.

3.2 Base de données : PostgreSQL

Le stockage des données est assuré par PostgreSQL, un système de gestion de bases de données relationnelles reconnu pour sa robustesse et ses performances.

Contrairement aux solutions basées sur des fichiers ou à certaines bases de données plus légères, PostgreSQL garantit une meilleure intégrité des données et offre des fonctionnalités avancées de gestion des transactions.

Les avantages principaux sont :

- Fiabilité élevée.
- Gestion efficace des données relationnelles.
- Support des transactions ACID.
- Excellentes performances sur les requêtes complexes.
- Utilisation fréquente dans les environnements professionnels.

La bibliothèque `psycopg` a été utilisée afin d'établir la communication entre Python et PostgreSQL.

3.3 Frontend : CustomTkinter

L'interface utilisateur a été développée avec CustomTkinter.

CustomTkinter est une extension moderne de Tkinter qui permet de créer des interfaces graphiques plus attrayantes tout en conservant la simplicité du framework d'origine.

Cette solution a été choisie pour les raisons suivantes :

- Apparence moderne et professionnelle.
- Facilité d'intégration avec Python.
- Compatibilité multiplateforme.
- Création rapide de fenêtres, formulaires et tableaux d'affichage.
- Personnalisation avancée des composants graphiques.

L'application utilise notamment des formulaires d'ajout et de modification, des cartes de présentation des livres ainsi qu'une interface dédiée au chatbot.

3.4 Intégration de l'intelligence artificielle avec Google Gemini

L'assistant conversationnel repose sur l'API Google Gemini.

Gemini est un modèle d'intelligence artificielle générative capable de comprendre le langage naturel et de produire des réponses pertinentes à partir d'un contexte fourni.

Son intégration permet à l'application de :

- Comprendre les questions formulées librement par l'utilisateur.
- Rechercher des informations dans le catalogue de la bibliothèque.
- Fournir des réponses contextualisées.
- Recommander des ouvrages selon les critères demandés.
- Améliorer l'expérience utilisateur grâce à une interaction naturelle.

Le chatbot ne se contente pas d'utiliser les connaissances générales du modèle. Il reçoit également les données provenant de la base de données afin de produire des réponses adaptées au contenu réel de la bibliothèque.

3.5 Gestion de configuration et sécurité

La gestion des paramètres sensibles a été réalisée à l'aide du package `python-dotenv`.

Les informations telles que :

- l'URL de connexion à la base de données ;
- la clé API Google Gemini ;
- les paramètres de configuration du projet ;

sont stockées dans un fichier `.env` séparé du code source.

Cette approche présente plusieurs avantages :

- Protection des informations sensibles.
- Meilleure portabilité du projet.
- Simplification du déploiement.
- Respect des bonnes pratiques de développement.

L'ensemble de ces technologies forme une architecture cohérente permettant de répondre efficacement aux exigences fonctionnelles et techniques du projet.

4. Architecture globale de l'application

L'application développée repose sur une architecture client-serveur modulaire visant à séparer clairement les responsabilités entre les différentes couches du système. Cette approche améliore la maintenabilité, la scalabilité ainsi que la lisibilité du code.

4.1 Vue d'ensemble de l'architecture

L'architecture globale de la solution est composée de trois couches principales :

- **Interface utilisateur (Frontend)** : développée avec *CustomTkinter* :contentReference[oaicite :0]index=0.
- **Backend (API REST)** : implémenté avec le framework *Flask* :contentReference[oaicite :1]index=1.
- **Base de données** : gérée via *PostgreSQL* :contentReference[oaicite :2]index=2.

Une couche supplémentaire d'intelligence artificielle est intégrée via l'API **Google Gemini** :contentReference[oaicite :3]index=3.

4.2 Description des composants

4.2.1 Frontend

L'interface utilisateur permet aux utilisateurs d'interagir avec le système. Elle offre :

- L'affichage du catalogue de livres.
- Les formulaires d'ajout et de modification.
- Une interface de recherche.
- Une interface dédiée au chatbot.

Le frontend communique avec le backend via des requêtes HTTP.

4.2.2 Backend

Le backend constitue le cœur logique de l'application. Il est développé avec Flask et expose une API REST permettant de gérer les opérations suivantes :

- Création de livres.
- Consultation du catalogue.
- Mise à jour des informations.
- Suppression d'ouvrages.
- Recherche avancée.

Le backend joue également le rôle d'intermédiaire entre le frontend, la base de données et le module d'intelligence artificielle.

4.2.3 Base de données

La persistance des données est assurée par PostgreSQL. La base contient principalement les tables suivantes :

- **books** : stockage des informations des livres.
- **users** (si applicable) : gestion des utilisateurs.
- **logs** (optionnel) : suivi des actions système.

La communication entre Python et la base de données est assurée par le driver *psycopg* : `contentReference[oaicite :4]index=4`.

4.2.4 Module d'intelligence artificielle

L'assistant intelligent est basé sur l'API **Google Gemini**.

Il permet :

- L'interprétation des requêtes en langage naturel.
- L'interrogation intelligente de la base de données.
- La génération de réponses contextuelles.
- La recommandation de livres.

4.3 Schéma architectural

Le flux global de l'application peut être résumé comme suit :

- L'utilisateur interagit avec l'interface **CustomTkinter**.

- Les requêtes sont envoyées à l'API **Flask**.
- Le backend traite la demande et interagit avec **PostgreSQL**.
- Si nécessaire, le backend interroge **Google Gemini**.
- La réponse est renvoyée au frontend puis affichée à l'utilisateur.

4.4 Conclusion de l'architecture

Cette architecture en couches permet une séparation claire des responsabilités, facilitant ainsi la maintenance et l'évolution du système. Elle garantit également une meilleure robustesse et une intégration fluide entre les différents composants.

5. Conception de la base de données

5.1 Modèle de données

La base de données est conçue autour d'une table principale **books** permettant de gérer le catalogue de la bibliothèque. Chaque livre possède des attributs décrivant ses caractéristiques ainsi que son état de disponibilité.

Le modèle adopté permet de couvrir les besoins essentiels du système : ajout, recherche, mise à jour et gestion des disponibilités.

5.2 Structure de la table **books**

Champ	Type	Description
id_livre	SERIAL	Identifiant unique auto-généré (clé primaire)
titre	TEXT	Titre du livre
auteur	TEXT	Auteur du livre
categorie	TEXT	Catégorie (roman, informatique, histoire...)
annee_publication	SMALLINT	Année de publication du livre
quantite_disponible	INTEGER	Nombre d'exemplaires disponibles
statut	TEXT	État du livre : disponible / emprunté / réservé
date_retour_prevue	DATE	Date prévue de retour en cas d'emprunt

5.3 Exemples de requêtes SQL utilisées

— Insertion d'un livre :

```
INSERT INTO books (titre, auteur, categorie, annee_publication,  
    quantite_disponible, statut)  
VALUES ('Le Petit Prince', 'Antoine de Saint-Exupry', 'Roman', 1943, 3, '  
    disponible');
```

— Recherche par auteur :

```
SELECT * FROM books WHERE auteur = 'Victor Hugo';
```

— Recherche par disponibilité :

```
SELECT * FROM books WHERE statut = 'disponible';
```

6. Fonctionnalités développées

6.1 Gestion CRUD des livres

L'application permet une gestion complète du catalogue de la bibliothèque :

- Ajout de nouveaux livres dans la base de données.
- Modification des informations existantes.
- Suppression de livres.
- Affichage de l'ensemble du catalogue.

6.2 Interface graphique

L'interface utilisateur développée avec **CustomTkinter** permet une interaction simple et intuitive. Elle est structurée en plusieurs fenêtres :

- tableau de bord principal,
- formulaire d'ajout,
- formulaire de modification,

7. Intégration du chatbot IA

7.1 Présentation du chatbot

Le chatbot est le composant intelligent du système. Il permet aux utilisateurs de poser des questions en langage naturel concernant les livres de la bibliothèque.

Il repose sur l'API **Google Gemini 2.5 Flash**, capable de comprendre les requêtes et de générer des réponses contextuelles.

7.2 Architecture du système conversationnel

Le fonctionnement du chatbot suit le flux suivant :

1. L'utilisateur saisit une question.
2. Le backend analyse la requête.
3. Une requête est envoyée à la base PostgreSQL.
4. Les données pertinentes sont extraites.
5. Un contexte est construit.
6. Ce contexte est envoyé à Gemini.
7. Une réponse est générée et renvoyée à l'utilisateur.

7.3 Communication avec la base de données

Le chatbot interagit directement avec les services backend afin d'extraire les informations nécessaires à la réponse.

7.4 Construction du prompt contextuel

Un prompt est construit dynamiquement afin d'améliorer la qualité des réponses :

You are an intelligent library assistant.

Your role:

- Answer only using the library catalog.
- Be friendly and professional.
- Format answers clearly using emojis and sections.
- Never invent books that are not in the catalog.
- If a book is unavailable, clearly indicate it.
- If no result exists, explain politely that nothing was found.

Response formatting rules:

For a single book:

Book Found

Title: ...

Author: ...

Category: ...

Year: ...

Status: ...

For multiple books:

Books Found

1.

...

...

2.

...

...

For recommendations:

Explain briefly why each book may interest the user.

Library Catalog:

{books}

User Question:

{question}

You are the official assistant of a smart library management system.

You help users:

- Search books
- Check availability
- Find books by author
- Recommend books by category
- Explain library information

If the answer is based on books from the catalog, always mention their status.

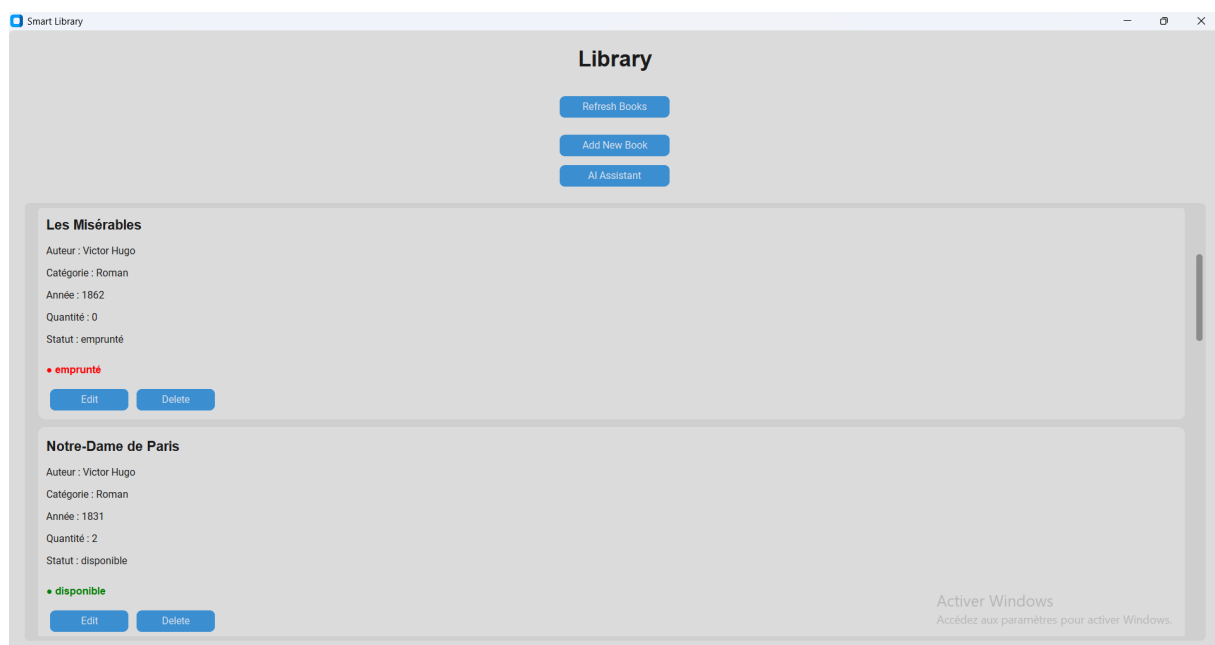
If a user asks for recommendations, recommend books from the catalog only.

7.5 Exemples de scénarios pris en charge

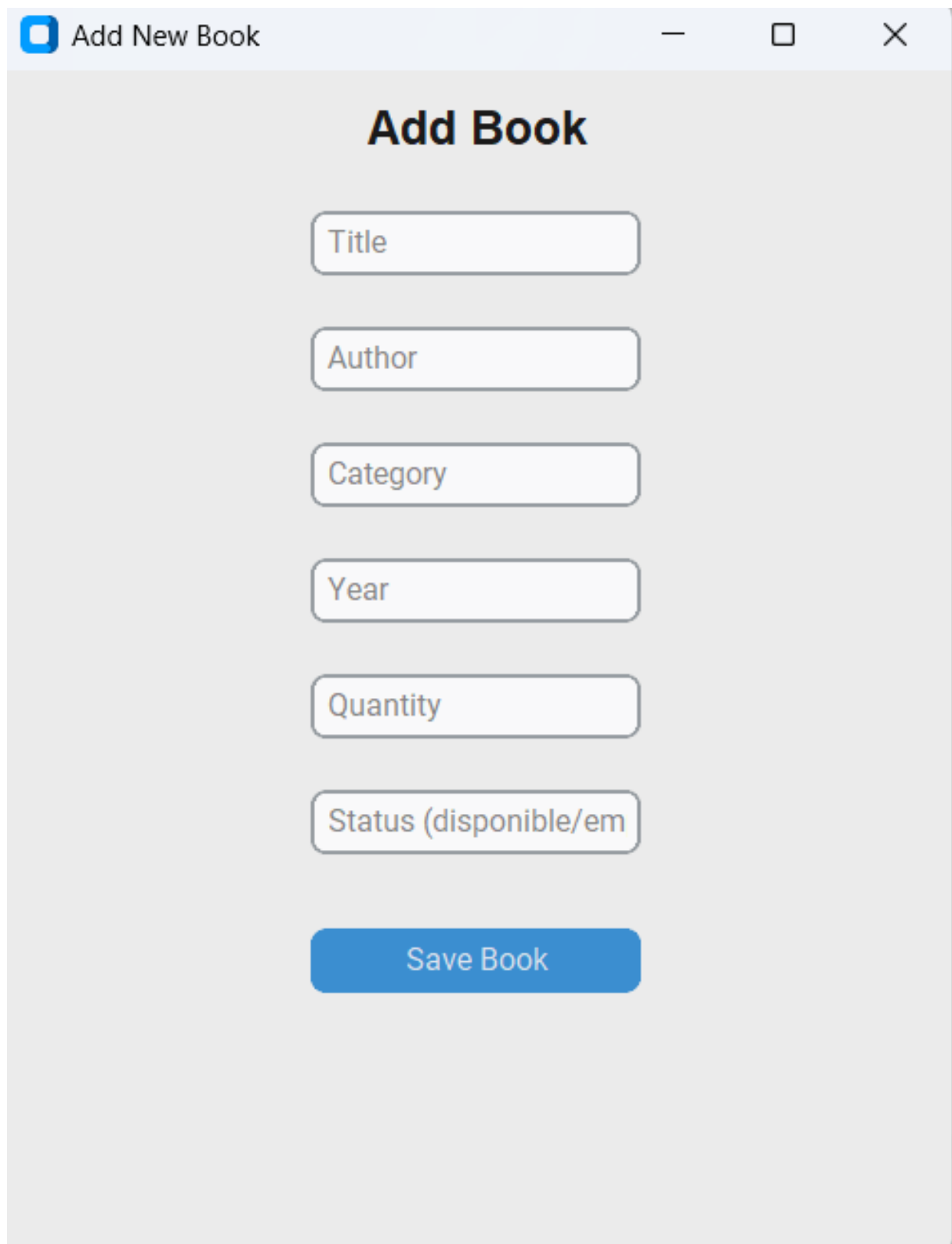
- Vérification de l'existence d'un livre par ID.
- Vérification de disponibilité.
- Recherche par auteur.
- Recommandations simples selon catégorie.

8. Captures d'écran et démonstration

8.1 Tableau de bord principal

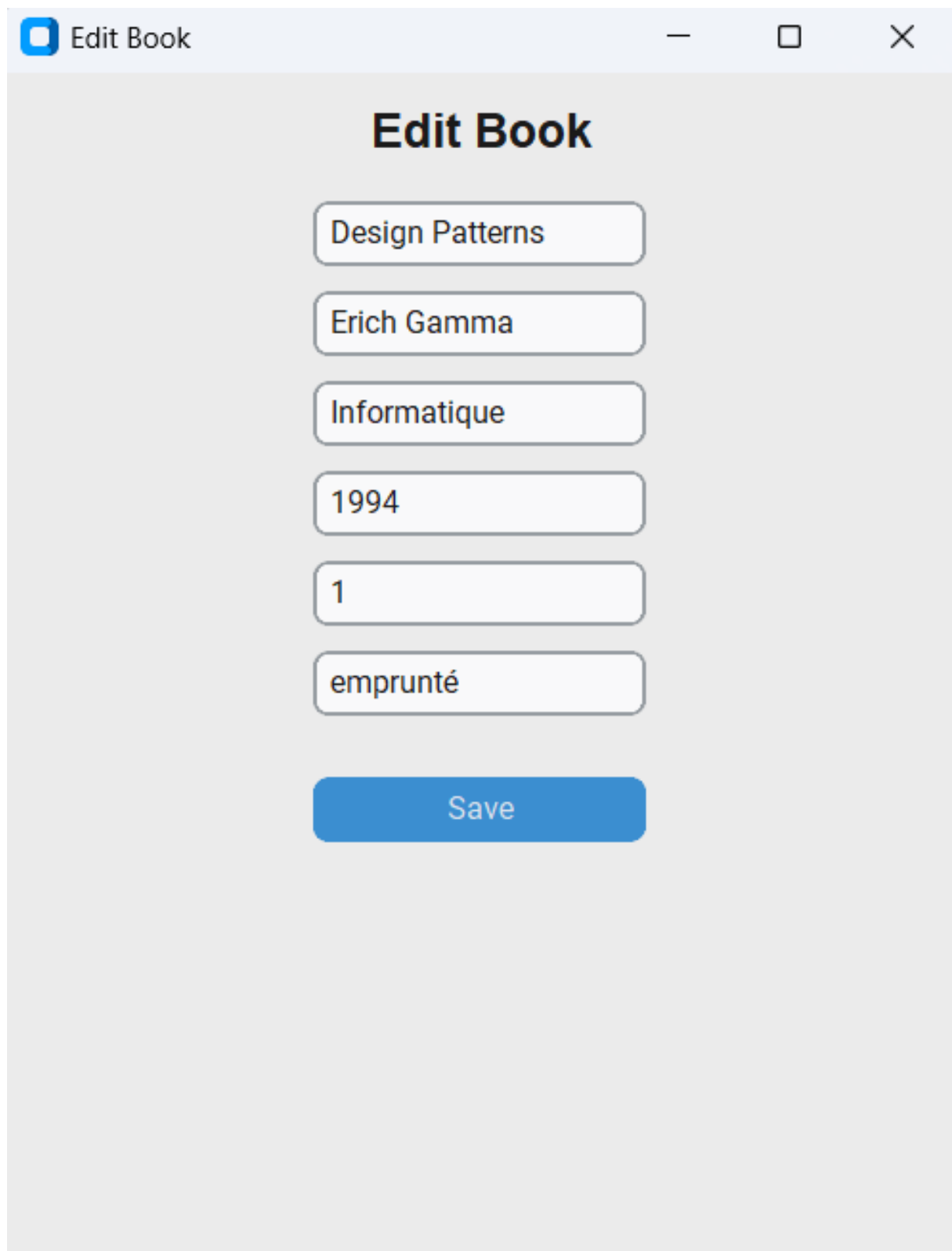


8.2 Ajout d'un livre



The image shows a web application window titled "Add New Book". Inside the window, there is a form titled "Add Book". The form contains six input fields, each with a label: "Title", "Author", "Category", "Year", "Quantity", and "Status (disponible/em)". Below these fields is a blue button labeled "Save Book".

8.3 Modification d'un livre



The screenshot shows a web application window titled "Edit Book". The window has a light blue header bar with the title and standard window controls (minimize, maximize, close). The main content area has a light gray background. At the top of the content area, the title "Edit Book" is displayed in a large, bold, black font. Below the title, there are six rounded rectangular input fields stacked vertically, each containing a book attribute: "Design Patterns", "Erich Gamma", "Informatique", "1994", "1", and "emprunté". At the bottom of the form, there is a prominent blue button with the text "Save" in white.

Edit Book

Design Patterns

Erich Gamma

Informatique

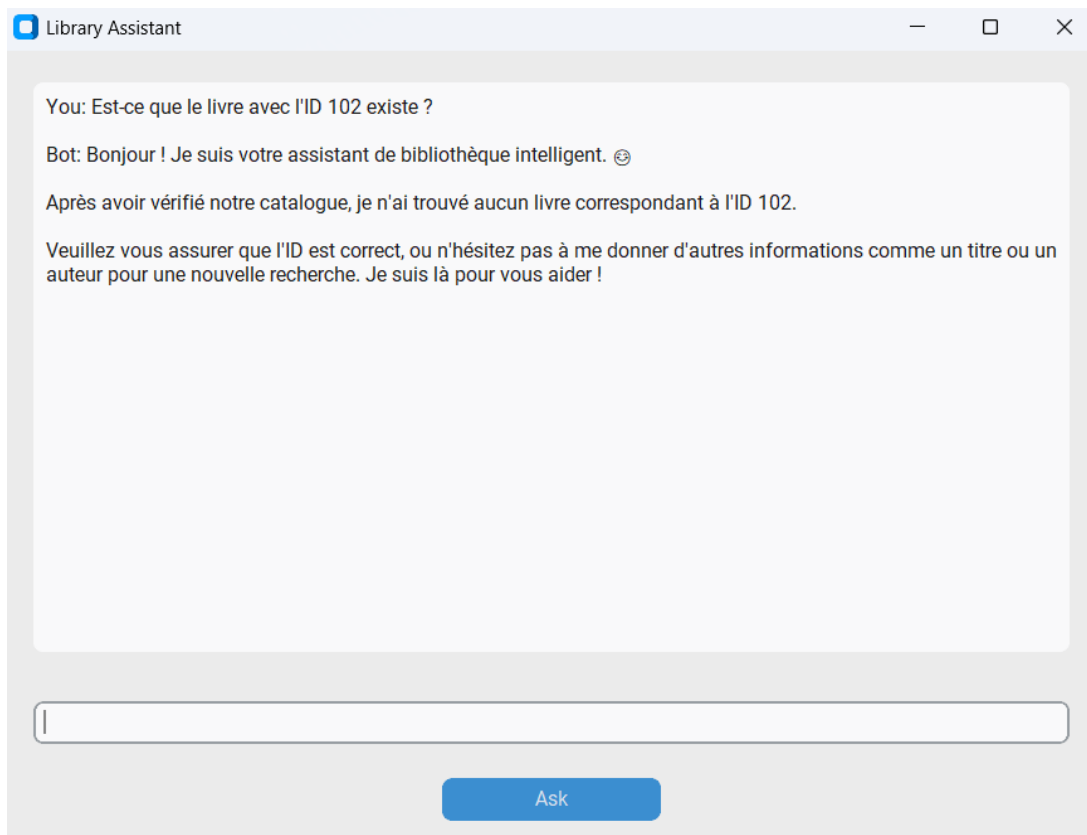
1994

1

emprunté

Save

8.4 Chatbot IA



```
(.venv) PS C:\Users\youne\Desktop\Library_project\backend> python test_chatbot.py
C:\Users\youne\Desktop\Library_project\backend\services\chatbot_service.py:2: FutureWarning:
All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:
https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

import google.generativeai as genai
Question: Est-ce que le livre avec l'ID 2 existe ?

Answer:
Bonjour ! 😊 Oui, j'ai trouvé un livre correspondant à l'ID 2 dans notre catalogue.

📖 Book Found

📖 Title: Les Misérables
👤 Author: Victor Hugo
📁 Category: Roman
📅 Year: 1862
📌 Status: emprunté
(.venv) PS C:\Users\youne\Desktop\Library_project\backend>
```


9. Conclusion

Ce projet a permis de développer une application complète de bibliothèque intelligente combinant gestion de données, interface utilisateur et intelligence artificielle.

L'utilisation de l'architecture MVC, de PostgreSQL et de l'API Google Gemini a permis de construire un système moderne, structuré et évolutif.

Ce projet constitue une base solide pour des extensions futures telles que la gestion des utilisateurs, des emprunts et le déploiement web.

10. Perspectives d'amélioration

- Ajout d'un système d'authentification.
- Gestion complète des emprunts et retours.
- Historique des actions utilisateur.
- Amélioration du chatbot avec mémoire conversationnelle.
- Déploiement web de l'application.
- Ajout de statistiques et dashboards analytiques.